https://www.techiedelight.com/?problem=TwoSum

parte 1
https://www.techiedelight.com/bit-hacks-part-1-basic/?fbclid=IwAR39rmIP1O5_GPgHBkOA8oleO3XFOKsU3bYwiZ39gpGksdQKzD1NdS53Arg
parte 2
https://www.techiedelight.com/bit-hacks-part-2-playing-kth-bit/
parte 3
https://www.techiedelight.com/bit-hacks-part-3-playing-rightmost-set-bit-number/
parte 4
https://www.techiedelight.com/bit-hacks-part-4-playing-letters-english-alphabet/
parte 5
https://www.techiedelight.com/bit-hacks-part-5-find-absolute-value-integer-without-branching/

https://www.techiedelight.com/bit-hacks-part-1-basic/?fbclid=IwAR39rmIP1O5_GPgHBkOA8oleO3XFOKsU3bYwiZ39gpGksdQKzD1NdS53Arg

Trucos de manipulación de bits

Pregunta	Expresión	Ejemplo
1- ¿ Comprueba si un entero es **par o impar** ?	& (AND) **N & 1** devuelve 1 o 0 dependiendo de si nes par o impar.	00010100 & (n = 20) 00000001 (1) ~~~~~~~~ 00000000 00010101 & (n = 21) 00000001 (1) ~~~~~~~~ 00000001

```
#include <iostream>
using namespace std;
int main()
{
   int n = 5;
   if (n & 1) {
      cout << n << " is odd";
   }
   else {
      cout << n << " is even";
   }
   return 0;
}
```

Nota:

- Tener en cuenta que el último bit(**bit más significativo** - **MSB**) de un binario dice 0=par, 1=impar
- Recuerda que c++, te brinda la posibilidad trabajar con números de dos formas:

1. **Como números normales**: los ve enteros, como 5, 12, 100, y hace operaciones normales (+, -, *, /).
2. **Como números binarios**: los ve como una fila de 0 y 1. Por tanto te permite hacer operaciones **bit a bit** (bitwise), que actúan sobre cada 0 o 1 individualmente, en lugar de usar el número completo. Entonces tu en este caso estas trabajando los números como binario, al usar operadores binario.

Pregunta	Expresión	Ejemplo
¿Detectar si dos números enteros tienen **signos opuestos** o no?	**^ (XOR)** **((x ^ y) < 0)**es negativa si xy ytienen signos opuestos **positivo**:bit más a la izquierda **0** **negativo:**bit más a la izquierda **1** **Entero de signos iguales:** XOR establece bit más a la izquierda **0** **Entero de signos opuestos:** XOR establece bit más a la izquierda **1**	**0**0…000100 ^ (x = 4) **0**0…001000 (y = 8) ~~~~~~~~~ **0**0…001100 igual signo 00…000100 ^ (x = 4) 11…111000 (y = -8) ~ ~~~~~~~~ **1**1…111100 diferente signo 10…000100 ^ (x = 4) 11…111000 (y = -8) ~ ~~~~~~~~ **0**1…111100 igual signo

```
#include <iostream>
#include <bitset>
using namespace std;

int main()
{
   int x = 4;
   int y = -8;

   //bitset<32>(x) convierte el entero x en una secuencia de 32 bits (0s y 1s).
   //cout << ... imprime en la consola ejemplo: 5 en binario es 00000000000000000000000000000101
   cout << x << " en binario es " << bitset<32>(x) << endl;
   cout << y << " en binario es " << bitset<32>(y) << endl;


   bool isOpposite = ((x ^ y) < 0);  // true si “x” e “y” tienen signo opuesto

   if (isOpposite) {
      cout << x << " and " << y << " tienen signo opuesto";
   }
   else {
      cout << x << " and " << y << " tienen signos iguales";
   }

   return 0;
}
```

La salida de la expresión `x ^ y` es **negativa** si `x` e `y` tienen **signos opuestos**. Sabemos que, para un número **positivo**, el **bit más a la izquierda(bit menos significativo – LSB) es** **0** y, para un número **negativo**, es **1**.

Ahora bien, para enteros de **signos iguales**, el operador XOR establecerá el bit más a la izquierda de la salida como **0**, y para enteros de **signo opuesto**, lo establecerá como **1**.

En C++, los números enteros negativos se representan usando **complemento a 2**. Esto significa que el **bit más a la izquierda** (el bit de mayor peso, también llamado MSB) se usa para indicar el signo:

- 0 → positivo
- 1 → negativo

Cuando haces un **XOR** (^) entre dos números:

1. Si un número es negativo y el otro positivo, el XOR dejará un 1 en el bit más significativo, porque los bits de signo son diferentes.
2. Si ambos son negativos o ambos positivos, el XOR no tendrá 1 en el bit de signo, porque son iguales y el XOR cancela los bits iguales.

Por eso, después de hacer x ^ y:

- Si el resultado tiene un 1 en el bit más a la izquierda ((x ^ y) < 0), eso significa que **los números tienen signos diferentes**, y si estabas comparando, el resultado será **menor que cero**.
- Si el resultado no tiene ese 1, los números tienen el mismo signo y el XOR no indica diferencia de signo.

Pregunta	Expresión	Ejemplo
Sumar 1 a un número entero **-~x**	~ (NOT): **Invierte** todos los **bits**: los 1 se vuelven 0 y viceversa. La **expresión -~x sumará 1 a un entero x.** Sabemos que para obtener el negativo de un número, se invierten sus bits y se le suma 1 (recuerde que los números negativos se almacenan en complemento a 2), es decir	Ejemplo: ~2 = -3 Ejemplo: -~2 = 3 00000010 ~ (x = 2) **1**1111101(C_2) (x = -3) **1**1111101(C_2) = 00000011($_{10}$) (x = -3)

```
#include <iostream>
using namespace std;
int main()
{
   int x = 4;
   cout << x << " + " << 1 << " is " << -~x << endl;
   x = -5;
   cout << x << " + " << 1 << " is " << -~x << endl;
   x = 0;
   cout << x << " + " << 1 << " is " << -~x << endl;
   return 0;
}
```

Salida:

```
4 en binario es 00000000000000000000000000000100
-8 en binario es 11111111111111111111111111111000
4 y -8 tienen signos opuestos
```

Ejemplo en binario (8 bits) — x = 5
X= 5_{10} = $(00000101)_2$
X= ~ 5_{10} = $(11111010)_2$

PROPIEDAD ~x= -x – 1
Convertir 5 a -5
5_{10} = $(00000101)_2$
Para −5:

- Invertimos los bits de 00000101 → 11111010
- Sumamos 1 → **11111011**
  Entonces **−5=11111011**$_2$.

Convertir 1 a -1

- 1_{10}=$(00000001)_2$

Para −1:

- Invertimos los bits de 00000001 → 11111110
- Sumamos 1 → **11111111**
  Entonces **−1=11111111**$_2$

~x= -5 – 1
11111011$_2$ --- es -5_{10}
+11111111$_2$ -- es -1_{10}

11111010$_2$ **es -6**$_{10}$

PROPIEDAD -~x= -(-x – 1)
-~x= x + 1

00000101$_2$ --- es 5_{10}
+ 00000001$_2$ -- es 1_{10}

00000110$_2$ **es 6**$_{10}$

-~x

```
4 + 1 es 5
-5 + 1 es -4
0 + 1 es 1
```

Qué hace -~x
Aplica una **identidad aritmética** que resulta en:
− ~ x=x+1

Pregunta	Expresión	Ejemplo

Intercambiar dos números sin usar ninguna tercera variable	x ^ x = 0. **x = x ^ y;** **y = x ^ y;** **x = x ^ y;** Este es uno de los trucos de bits más sencillos y comunes. La idea es usar operadores XOR para intercambiar dos números según su propiedad x ^ x = 0	00010100 ^ (x = 20) 00010101 (y = 21) ~~~~~~~~ 00000001 **x = x ^ y;** **(diferencia)** 00000001 ^ (x = 1) 00010101 (y = 21) ~~~~~~~~ 00010100 **y = x ^ y;** 00000001 ^ (x = 1) **00010100** (y = 21) ~~~~~~~~ **00010101** **x = x ^ y;**

```
#include <iostream>
using namespace std;

// Función para intercambiar dos variables usando XOR
void swap(int &x, int &y)
{
   // Solo se realiza el intercambio si los valores son diferentes
   if (x != y)   {
      x = x ^ y; // Primer paso: x se convierte en x ^ y, haya la diferencia
      y = x ^ y; // Segundo paso: obtenemos y
      x = x ^ y; // Tercer paso:  obtenemos x
   }
}
int main()
{
   int x = 3, y = 4;
   // Imprime valores antes del intercambio
   cout << "Antes de intercambio: x = " << x << " y  y = " << y;

   // Llama a la función swap para intercambiar los valores
   swap(x, y);

   // Imprime valores después del intercambio
   cout << "\nDespues de intercambio: x = " << x << " y y = " << y;
   return 0;
}
```

https://www.techiedelight.com/bit-hacks-part-2-playing-kth-bit/

Pregunta	Expresión	Ejemplo

Desactiva el k-ésimo bit en un número	<< (desplaza izquierda) a << b Desplaza a la izquierda**(a)** un número especificado de bits**(b)**. 1 << (3-1) 0000000**1** << 2 = **0000010****0** **00000100** ~ (x = **4**) **11111011**(C_2) n **&** ~(1 << (k - 1))	n = 20y k-ésimo = 3 0001010**0** & (n = 20) 11111011 ~ (1 << (3-1)) ~~~~~~~~ 0001**0**0000

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para apagar (poner en 0) el k-ésimo bit de `n`
int turnOffKthBit(int n, int k) {
   return n & ~(1 << (k - 1)); // Se crea una máscara con un 1 en la posición k, se invierte (~) y se aplica AND
}

int main()
{
   int n = 20; // Número inicial
   int k = 3;  // Bit a apagar (contando desde la derecha, comenzando en 1)

   cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra n en binario
   cout << "Apagando el bit k-esimo...\n";                 // Mensaje en español
   n = turnOffKthBit(n, k);                                // Apaga el k-ésimo bit de n
   cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el nuevo valor en binario

   return 0; // Fin del programa
}
```

Explicación de n & ~(1 << (k - 1));
(1 << (k – 1): Se crea una máscara con un 1 en la posición k y Desplaza a la izquierda el 1 un número de **k-1** de bits
~ (1 << (k – 1) Se invierte la mascara, por ejemplo si la mascara era **00000100, ahora queda 11111011**
n & ~(1 << (k – 1)): numero le hace un and con la mascara y claro se va a desactivar solo donde esta **0**,
Ejemplo: n= 0001 0**1**00
~ (1 << (k – 1)= 1111 1011
resultado: = 0001 0000
La idea es usar los operadores bit a bit **<<**, **&** y **~**. Usando la expresión ~ (1 << (k - 1)), obtenemos un número con todos sus bits activados, excepto el k'thbit. Si realizamos una operación AND bit a bit de esta expresión con n, es decir n & ~(1 << (k - 1)), obtenemos un número con todos sus bits iguales n excepto el k'thbit, que se pondrá a 0.
Por ejemplo, considere n = 20 y k = 3.
0001 0100 y (n = 20)
1111 1011 ~ (1 << (3-1))
~~~~~~~~
0001 0000

Pregunta	**Expresión**	**Ejemplo**
~~~~~~~~

Activa el k-ésimo bit en un número	1 << (3-1) 00000001 << 2 = **00000100** n | (1 << (k - 1))	n = 20y k-ésimo = 3 00010100 | (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00011100

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para encender (poner en 1) el k-ésimo bit de `n`
int turnOnKthBit(int n, int k) {
    return n | (1 << (k - 1)); // Se crea una máscara con un 1 en la posición k y se aplica OR
}

int main()
{
    int n = 20; // Número inicial
    int k = 4;  // Bit a encender (contando desde la derecha, empezando en 1)

    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra n en binario
    cout << "Encendiendo el bit k-esimo...\n";              // Mensaje en español
    n = turnOnKthBit(n, k);                                 // Enciende el k-ésimo bit de n
    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el nuevo valor en binario

    return 0; // Fin del programa
}
```

Explicación de n | (1 << (k - 1));
(1 << (k – 1): Se crea una máscara con un 1 en la posición k y Desplaza a la izquierda el 1 un número de **k-1** de bits
n | (1 << (k – 1)): numero le hace un OR con la mascara y claro se va a **activar** solo donde esta **1**,
Ejemplo: n= 0001 **0**100
| (1 << (k – 1)= 0000 1000
resultado: = 0001 1100

La idea es usar los operadores << y bit a bit |. Usando la expresión **1 << (k - 1)**, obtenemos un número con todos los bits a 0, excepto el k'thbit. Si realizamos un análisis bit a bit **OR** de esta expresión con n, es decir, **n | (1 << (k - 1))**, obtenemos un número con todos los bits iguales nexcepto el k'thbit, que se establecerá en 1.
Por ejemplo, considere n = 20 y k = 4.
00010100 | (n = 20)
00001000 (1 << (4 – 1))
~~~~~~~~
00011100
~~~~~~~~

Pregunta	Expresión	Ejemplo
Comprueba si el k-ésimo bit está establecido, es decir activo o 1	n & (1 << (k – 1)) != 0	20 en binario es 00010100, el bit k=3 está activado

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para comprobar si el k-ésimo bit de `n` está encendido (1) o no
bool isKthBitSet(int n, int k) {
   return (n & (1 << (k - 1))) != 0; // Aplica una máscara en la posición k y verifica si es distinto de 0
}

int main()
{
   int n = 20; // Número inicial
   int k = 3;  // Bit a verificar (contando desde la derecha, empezando en 1)

   cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra n en binario

   if (isKthBitSet(n, k)) { // Verifica si el k-ésimo bit está encendido
      cout << "El bit k-ésimo está ENCENDIDO (1)";       // Mensaje en español si está encendido
   }
   else {
      cout << "El bit k-ésimo está APAGADO (0)";         // Mensaje en español si está apagado
   }

   return 0; // Fin del programa
}
```

La idea es usar los operadores << y bit a bit &. Usando la expresión 1 << (k - 1), obtenemos un número con todos los bits a 0, excepto el k'thbit. Si realizamos operaciones bit a bit AND de esta expresión con n, es decir, n & (1 << (k - 1)), cualquier valor distinto de cero indica que su k'thbit está activo.

Por ejemplo, considere n = 20 y k = 3.
00010100 & (n = 20)
00000100 (1 << (3-1))
~~~~~~~~
00000100 valor distinto de cero
~~~~~~~~

Pregunta	Expresión	Ejemplo
Invertir el k-ésimo bit	n ^ (1 << (k – 1))	n = 20y k-ésimo = 3 00010100 ^ (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00011100 00011100 ^ (n = 20) 00001000 (1 << (3-1)) ~~~~~~~~ 00010100

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para alternar (toggle) el k-ésimo bit de `n`
int toggleKthBit(int n, int k) {
    return n ^ (1 << (k - 1)); // Se crea una máscara con un 1 en la posición k y se aplica XOR para invertir el bit
}

int main()
{
    int n = 20; // Número inicial
    int k = 3;  // Bit a invertir (contando desde la derecha, empezando en 1)

    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra n en binario
    cout << "Alternando (toggle) el bit k-ésimo de n...\n";  // Mensaje en español
    n = toggleKthBit(n, k);                                  // Alterna el k-ésimo bit de n

    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el nuevo valor en binario

    return 0; // Fin del programa
}
```

La idea es usar los operadores ^ y << bit a bit. Al usar la expresión **1 << (k - 1)**, obtenemos un número con todos los bits a 0, excepto el k-ésimo bit. Si realizamos una operación **XOR** bit a bit de esta expresión con n, es decir, **n ^ (1 << k)**, podemos alternar fácilmente su k-ésimo bit como 0 ^ 1 = 1 y 1 ^ 1 = 0.

https://www.techiedelight.com/bit-hacks-part-3-playing-rightmost-set-bit-number/

Desactivación del bit establecido más a la derecha de un número

Pregunta	Expresión	Ejemplo
¿Desactivar el bit establecido más a la derecha de un número?	n & (n-1)	00010100 & (n = 20) 00010011 (n-1 = 19) ~~~~~~~~ 00010000

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para apagar (poner en 0) el bit encendido más a la derecha de `n`
unsigned unsetRightmostSetBit(unsigned n)
{
        return n & (n - 1); // Al restar 1, el bit encendido más a la derecha pasa a 0 y los bits siguientes a 1.
                            // Con AND se apaga ese bit y los bits siguientes
}

int main()
{
        int n = 20; // Número inicial

        cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el número en binario

        cout << "Apagando el bit encendido más a la derecha de " << n << "...\n"; // Mensaje en español
        n = unsetRightmostSetBit(n); // Llama a la función para apagar el bit encendido más a la derecha

        cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el resultado en binario

        return 0; // Fin del programa
}
```

La expresión **n & (n-1)** desactivará el bit establecido más a la derecha de un número n. La expresión n-1 invertirá todos los bits después del bit establecido más a la derecha de n (incluido el bit establecido más a la derecha). Por lo tanto, n & (n-1)resultará en el último bit invertido de n. Considere,

1. **Resta directa con préstamos (igual que en decimal)**
Se hace columna por columna de derecha a izquierda:

- 0−0=0
- 1−0=1
- 1−1=0
- 0−1=1 (pero aquí necesitas pedir **un préstamo** del bit más a la izquierda).

```
  0001 0100   (20)  (n-1) // Al restar 1, el bit encendido más a la derecha pasa a 0 y los bits siguientes a 1.
-  0000 0001   ( 1)
------------
  0001 0011   (19)

   0001 0100   (20)
 & 0001 0011   (19)   (n-1)
    ------------
    0001 0000   (16)
```

Pregunta	Expresión	Ejemplo
Comprueba si un entero positivo es una **potencia de 2** sin usar ninguna bifurcación o bucle.	Si al desactivar el único bit, da 0 entonces es potencia de 2, porque Si el número es una potencia de 2, solo tiene un solo bit establecido n & (n-1) == **0** El **0** se considera incorrectamente como una potencia de 2 aquí. Para remediar esto, use: **n && !(n & (n - 1));**	00010000 & (n = 16, solo un bit establecido) 00001111 (n-1 = 15) ~~~~~~~~ 00000000

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para comprobar si `n` es potencia de 2
bool isPowerOfTwo(unsigned n)
{
   // Muestra `n` en binario
        cout << n << " en binario es " << bitset<8>(n) << "\n";
   // Muestra `n - 1` en binario
        cout << (n - 1) << " en binario es " << bitset<8>(n - 1) << "\n\n";

   // Si `n` es potencia de 2, solo tiene un bit en 1
   // La operación n & (n - 1) apaga ese único bit
        return !(n & (n - 1)); // Devuelve true si n es potencia de 2, false si no
}

int main()
{
        unsigned n = 16; // Número a verificar

        if (isPowerOfTwo(n)) { // Llama a la función
                cout << n << " es potencia de 2";
        } else {
                cout << n << " NO es potencia de 2";
        }

        return 0; // Fin del programa
}
```

Como se mencionó anteriormente, la expresión **n & (n-1)** deshabilitará el bit establecido más a la derecha de un número. Si el número es una potencia de 2, solo tiene 1 bit establecido y n & (n-1) deshabilitará el único bit establecido. Por lo tanto, podemos decir que n & (n-1) devuelve 0 si n es una potencia de 2; de lo contrario, no es una potencia de 2.
00010000 & (n = 16, solo un bit establecido)
00001111 (n-1 = 15)
~~~~~~~~
00000000

Pregunta	Expresión	Ejemplo
~~~~~~~~

Encuentre la posición del bit establecido más a la derecha	**n ^ (n & (n-1))**
	
	(n & (n-1): Desactivo el bit establecido más a la derecha
	
	n ^ (n & (n-1)): vuelvo a recuperar con xor **solo** el bit establecido más a la derecha.
	
	int pos = 0;
	while (n != 0)
	{
	n = n >> 1;
	pos++;
	}
	
	return pos;
	
	Otra forma sería
	(n & -n) == n
	
	Return Math.log2(resultado) + 1;
		00010100 & (n = 20)
		00010011 (n-1 = 19)
		~~
		00010000
		
		00010100 ^ (n = 20)
		00010000
		~~
		00000100 vuelvo a recuperar con xor **solo** el bit establecido más a la derecha. **Ahora** el bit establecido más a la derecha nos lo dirá la posición
		
		Otra forma sería
		00…0010100 & (n = 20)
		11…1101100(C_2) (-n = -20)
		~~
		00…0000100

```
#include <iostream>
#include <bitset>
using namespace std;

// Devuelve la posición del bit encendido más a la derecha de `n`
int positionOfRightmostSetBit(int n)
{
   // Si el número es impar, el bit más a la derecha está en la posición 1
   if (n & 1) {
      return 1;
   }

   // Apaga todos los bits excepto el más a la derecha que está encendido
   n = n ^ (n & (n - 1));

   // Encuentra la posición del único bit en 1
   int pos = 0;
   while (n)  //cuando se haga cero ya parara el ciclo. En C++ cualquier valor distinto de 0 se considera true
   {
      n = n >> 1; // Desplaza los bits a la derecha
      pos++;      // Incrementa la posición
   }
   return pos; // Retorna la posición del bit encendido más a la derecha
}

int main()
{
   int n = 20; // Número a analizar
```

```
    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el número en binario
    cout << "La posición del bit encendido más a la derecha es "
         << positionOfRightmostSetBit(n);                  // Muestra la posición

    return 0; // Fin del programa
}
```

Una forma mas rapida sin el ciclo seria Se puede usar **logaritmo base 2**:

```
#include <cmath> // para log2
int positionOfRightmostSetBit(int n)
{
    if (n & 1) return 1;          // Si el número es impar, el bit más a la derecha está en 1
    n = n ^ (n & (n - 1));        // Aísla el bit encendido más a la derecha
    return log2(n) + 1;           // log2(n) da la posición del bit (empezando desde 0), sumamos 1
}
```

otra forma

```
#include <iostream>
#include <math.h>
using namespace std;

// Devuelve la posición del bit encendido más a la derecha de `n`
int positionOfRightmostSetBit(int n)
{
    if (n & 1) {              // Si el número es impar, el bit más a la derecha está en la posición 1
        return 1;             // Retorna 1 directamente
    }

    return log2(n & -n) + 1;   // n & -n aísla el bit encendido más a la derecha
                               // log2() devuelve su posición (empezando desde 0), se suma 1 para que empiece desde 1
}

int main()
{
    int n = 20; // Número a analizar

    cout << "La posición del bit encendido más a la derecha es "
         << positionOfRightmostSetBit(n); // Muestra el resultado en español

    return 0; // Fin del programa
}
```

La idea es anular el bit más a la derecha del número n y aplicar una operación XOR al resultado con n. Entonces, el bit más a la derecha n será la posición del único bit activo en el resultado. Tenga en cuenta que si n es impar, podemos devolver directamente 1, ya que el primer bit siempre está activo para los números impares.
Por ejemplo, el número 20 en binario es 00010100, y la posición del bit establecido más a la derecha es 3.

Pregunta	Expresión	Ejemplo

Encuentra la posición del único bit establecido en un número	**Resultado =** n ^ (n & (n-1)) **Return Math.log2(resultado) + 1;**	00010000 & (n = 16) 00001111 (n-1 = 15) ~~~~~~~~ 00000000 log2(16) + 1 = 5 **Otra forma sería** 00…0010100 & (n = 20) 11…1101100(C_2) (-n = -20) ~~~~~~~~~~ 00…0000100

```
#include <iostream>
#include <bitset>
#include <math.h>
using namespace std;

// Devuelve la posición del único bit encendido de `n`
int positionOfSetBit(int n)
{
   // Si el número tiene más de un bit en 1, muestra mensaje de error
   if (n & (n - 1)) // Desapaga el bit más a la derecha y verifica si queda algún otro
   {
      cout << "Entrada incorrecta: más de un bit está encendido";
      return 1;     // Retorna 1 por defecto en caso de error
   }

   return log2(n) + 1; // log2(n) devuelve la posición del bit (empezando desde 0), sumamos 1
}

int main()
{
   int n = 16; // Número a analizar

   cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el número en binario
   cout << "La posición del único bit encendido es " << positionOfSetBit(n); // Muestra la posición

   return 0; // Fin del programa
}
```

La idea es anular el bit más a la derecha del número ny comprobar si se convierte en 0. Si no es cero, sabemos que hay otro bit activo y que la entrada es inválida. Si se convierte en 0, podemos encontrar la posición del único bit activo procesando cada bit de nuno en uno o usando directamente log2(n) + 1. Por ejemplo, el número 16 en binario es 00010000, y la posición del bit establecido más a la derecha es 5.

Pregunta	Expresión	Ejemplo

Cálculo de la paridad de un número
Número total de 1 en un número binario
• La paridad **impar** (1) significa un número impar de 1
• La paridad **par** (0) significa un número par de 1.
	paridad = false;
	while (n != 0) {
	//invertir bandera paridad
	paridad = !paridad;
	// apagando bit establecido derecha
	n = n & (n - 1);
	}
	
	return paridad	

Una solución sencilla es calcular la paridad **comprobando cada bit n uno por uno**. El tiempo empleado es proporcional al número total de bits del número.

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para encontrar la paridad de un número `n`
// Devuelve true si la cantidad de bits en 1 es impar, false si es par
bool findParity(unsigned n)
{
    bool parity = false; // Inicializamos paridad en false (par)

    // Se ejecuta hasta que n se vuelva 0
    while (n)
    {
        if (n & 1) {       // Si el bit más a la derecha está en 1
            parity = !parity; // Invertimos la paridad
        }

        n = n >> 1;        // Desplazamos n un bit a la derecha (equivale a dividir entre 2)
    }

    return parity;         // Devuelve true (impar) o false (par)
}

int main()
{
    unsigned n = 31; // Número a analizar

    cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el número en binario

    if (findParity(n)) {
```

```
    cout << "La paridad de " << n << " es impar";        // Mensaje en español si la paridad es impar
  }
  else {
    cout << "La paridad de " << n << " es par";          // Mensaje en español si la paridad es par
  }

  return 0; // Fin del programa
}
```

Podemos obtener un **mejor rendimiento desactivando el bit establecido más a la derecha del número, uno por uno, y contando la paridad**. El siguiente código utiliza un enfoque similar al conteo de bits de **Brian Kernighan**. El tiempo que tarda es proporcional al número total de bits establecidos.

```
#include <iostream>
#include <bitset>
using namespace std;

// Función para encontrar la paridad de un número `n`
// Devuelve true si la cantidad de bits en 1 es impar, false si es par
bool findParity(unsigned n)
{
  bool parity = false; // Inicializamos la paridad en false (par)

  // Se ejecuta hasta que n se vuelva 0
  while (n)
  {
    parity = !parity;       // Invertimos la paridad cada vez que encontramos un bit en 1

    n = n & (n - 1);        // Apagamos el bit encendido más a la derecha de n
                            // Esto reduce el número de iteraciones al número de bits en 1
  }

  return parity;            // Devuelve true (impar) o false (par)
}

int main()
{
  unsigned n = 31; // Número a analizar

  cout << n << " en binario es " << bitset<8>(n) << endl; // Muestra el número en binario

  if (findParity(n)) {
    cout << "La paridad de " << n << " es impar";       // Mensaje si la paridad es impar
  }
  else {
    cout << "La paridad de " << n << " es par";         // Mensaje si la paridad es par
  }

  return 0; // Fin del programa
}
```

La "paridad" se refiere al número total de unos en un número binario. La paridad impar (1) significa un número impar de unos, y la paridad par (0) significa un número par de unos. Los bits de paridad se utilizan a menudo como un método sencillo para detectar errores durante la transmisión y recepción de datos digitales.

Sumando +1 y restando -1

Cual es la diferencia de un número con su antecesor?
La diferencia clave entre un número n y su antecesor inmediato n-1 está en **cómo cambian los bits cuando restamos 1** en binario.
n-1:
- tendrá todos los **bits invertidos después del bit establecido más a la derecha de n** (incluido el bit establecido más a la derecha).
- Tendrá todos los **bits iguales antes del bit establecido** más a la derecha de n

📌 Reglas generales:

1. **Antes del bit activo más a la derecha en n:** → Los bits permanecen **igualitos** en n y en n-1.
2. **El bit activo más a la derecha en n:** → Se **convierte** en **0** en n-1.
3. **Los bits a la derecha de ese bit activo más a la derecha:** → Se vuelven **todos 1** en n-1.

Ejemplo:
n = **0010** **1**000
- 1 = 0000 0001
n - 1 = **0010** **0**111

n - 1 apaga el bit encendido más a la derecha de n y enciende todos los bits a su derecha.

Cual es la diferencia de un número con su predecesor?
n+1:
- tendrá **todos los bits iguales antes del bit "NO establecido"** más a la derecha de n
- tendrá **todos los bits invertidos después del bit "NO establecido"** más a la derecha de n (incluido el bit "NO establecido" más a la derecha).

📌 Reglas para n + 1:

1. **Antes del bit "0" más a la derecha en n:** → Todos los bits permanecen **iguales** en n y en n + 1.
2. **Ese bit "0" más a la derecha en n:** → Se convierte en **1** en n + 1.
3. **Todos los bits a la derecha de ese "0":** → Se vuelven **0** en n + 1.

Ejemplo:
n =**0001** **0**111
+1 =0000 0001
n + 1 =**0001** **1**000

Jugando con las letras del alfabeto

Si cogemos todos los caracteres en mayúsculas (A-Z) y todos los minúsculas (a-z) e imprimimos su código en binario, observamos que **todos los bits son iguales**, **excepto el sexto bit** (contando desde la derecha):
a = 1100001 b = 1100010 c = 1100011 d = 1100100 e = 1100101 f = 1100110 g = 1100111 h = 1101000 i = 1101001 j = 1101010 k = 1101011 l = 1101100 m = 1101101 n = 1101110 o = 1101111 p = 1110000 q = 1110001 r = 1110010 s = 1110011 t = 1110100 u = 1110101 v = 1110110 w = 1110111 x = 1111000 y = 1111001 z = 1111010

A = 1000001 B = 1000010 C = 1000011 D = 1000100 E = 1000101 F = 1000110 G = 1000111 H = 1001000 I = 1001001 J = 1001010 K = 1001011 L = 1001100 M = 1001101 N = 1001110 O = 1001111 P = 1010000 Q = 1010001 R = 1010010 S = 1010011 T = 1010100 U = 1010101 V = 1010110 W = 1010111 X = 1011000 Y = 1011001 Z = 1011010

Minúsculas: tienen el sexto bit a **1** a = 1100001
Mayúsculas: tienen el sexto bit a **0** . A = 1000001

Por tanto para convertir de **Minúsculas a Mayúsculas** o viceversa solo tenemos que cambiar el 6to bits, **Haciendo un XOR** de un carácter con el **carácter espacio** cuyo código es el **32 en decimal(128; 64; 32; 16; 8; 4; 2; 1),** por lo tanto 100000 en binario podemos hacer la conversión, ya que esta operación **invertirá** el bit **6**.

Pregunta	**Expresión**	**Ejemplo**
Invertir minúsculas y mayúscula	letra ^ (1 << 5)	110 0001 ^ (a) 010 0000 (6 bit) ---------- 100 0001 (A) 100 0001 ^ (A) 010 0000 (6 bit) ---------- 110 0001 (a)

```
#include <iostream>
using namespace std;

int main() {
   char c;

   cout << "Ingresa una letra: ";
   cin >> c;
```

```
    // Invertir mayúscula/minúscula
    char invertida = c ^ (1 << 5);  //el 6to carácter lo invierto

    cout << "La letra invertida es: " << invertida << endl;

    return 0;
}
```

Ese truco con `^ (1 << 5)` es **muy usado en ASCII** porque:

- `a–z` (minúsculas) → difieren de `A–Z` (mayúsculas) solo en el **bit 5**.
- Si hacemos XOR con `1 << 5` (o sea `32` decimal), invertimos ese bit y cambiamos entre **mayúscula ↔ minúscula**.

Pregunta	**Expresión**	**Ejemplo**
Convierte caracteres en mayúsculas a **minúsculas**.	letra | (1 << 5)	1100001 | (a) 0100000 (6 bit) ---------- **1100001 (a)** 1000001 | (A) 0100000 (6 bit) ---------- **1100001 (a)**

Podemos convertir fácilmente un carácter en mayúscula en un carácter en minúscula correspondiente tomándolo **bit a bit OR** con un **espacio (valor 32)**.

```
// Convertir carácter mayúsculas a minúsculas
for (char ch = 'A'; ch <= 'Z'; ch++) {
    cout << char(ch | ' '));        // prints abcdefghijklmnopqrstuvwxyz
}
```

otra forma

```
#include <iostream>
using namespace std;

int main() {
    char c;

    cout << "Ingresa una letra en MAYÚSCULA: ";
    cin >> c;
```

```
    // Convertir a minúscula
    char minuscula = c | (1 << 5);

    cout << "En minúscula es: " << minuscula << endl;

    return 0;
}
```

Para **convertir mayúsculas a minúsculas** en ASCII se aprovecha que:

- A = 65 (01000001_2)
- a = 97 (01100001_2)

La **única diferencia** es el **bit 5** (valor 32).

👉 Si lo encendemos con **OR (|)** → siempre obtenemos minúscula.

Pregunta	Expresión	Ejemplo
Convierte caracteres en minúsculas a **mayúsculas**.	letra **&** ~(1 << 5)	0100000 (1 << 5) 1011111 ~(1 << 5) 1100001 **&** (a) 1011111 ~(1 << 5) ---------- **1000001 (A)** 1000001 **&** (A) 1011111 ~(1 << 5) ---------- **1000001 (A)**

De manera similar, podemos convertir fácilmente un carácter en minúscula en un carácter en mayúscula correspondiente tomándolo bit a bit ANDcon un carácter de guión bajo.

```
// Convertir carácter minúsculas a mayúsculas
for (char ch = 'a'; ch <= 'z'; ch++) {
    cout << char(ch & '_'));    // prints ABCDEFGHIJKLMNOPQRSTUVWXYZ
}
```

otra forma

```
#include <iostream>
using namespace std;

int main() {
    char c;
```

```
    cout << "Ingresa una letra en minúscula: ";
    cin >> c;

    // Convertir a mayúscula
    char mayuscula = c & ~(1 << 5);

    cout << "En mayúscula es: " << mayuscula << endl;

    return 0;
}
```

Ahora hacemos la conversión inversa: de **minúsculas → mayúsculas**.
En ASCII:

- a = 97 = **01100001_2**
- A = 65 = **01000001_2**

👉 La diferencia sigue estando en el **bit 5** (valor 32).

- Para asegurarnos de que quede en **mayúscula**, **apagamos** ese bit con **AND (&)** usando la máscara ~(1 << 5).

Pregunta	**Expresión**	**Ejemplo**
Encuentra la posición de una letra en el alfabeto	letra **&** 31	**1000001** **& (A)** 0011111 (31) ~~~~~~~ 0000001 (1) 1100100 **&** (d) 0011111 (31) ~~~~~~~ 0000100 (4)

```
#include <iostream>
using namespace std;

int main() {
    char c;

    cout << "Ingresa una letra: ";
    cin >> c;

    int pos = c & 31; // Obtiene la posición [1–26]

    cout << "La letra '" << c << "' está en la posición " << pos << " del alfabeto." << endl;

    return 0;
}
```

Podemos encontrar fácilmente la posición de una letra `[1-26]` en el alfabeto calculándola bit a bit **AND** con ASCII 31 (`00011111`en binario). El uso de mayúsculas y minúsculas es irrelevante.

📌 ¿Por qué funciona con 31 (00011111_2)?

Porque obtenemos solo los **5 bits menos significativos**, que corresponden al rango 1–26.

- 'A' (65) → 01000001_2 & 00011111_2 = 00000001_2 = 1
- 'C' (67) → 01000011_2 & 00011111_2 = 00000011_2 = 3
- 'a' (97) → 01100001_2 & 00011111_2 = 00000001_2 = 1
- 'z' (122) → 01111010_2 & 00011111_2 = 00011010_2 = 26

https://www.techiedelight.com/bit-hacks-part-5-find-absolute-value-integer-without-branching/

Pregunta	Expresión	Ejemplo

Dado un número entero, calcule su valor absoluto (abs) sin ramificar(Es decir **sin usar estructuras condicionales** como if, ?:, switch, etc)
 	
Aquí **sí hay ramificación**, porque el procesador debe decidir entre dos caminos distintos según el signo de n.
 	
`if (n < 0)`
`  return -n;`
`else`
`  return n;`
		Positivo
		00000110 (n = 6)
		00000000 (máscara = 0)
		
		00000110 ^ (n + máscara)
		00000000 (máscara)
		~~
		00000110 abs(n)
		
		negativo
		11111010 (n = -6)
		11111111 (máscara = -1)
		
		11111001 ^ (n + máscara)
		11111111 (máscara)
		~~
		00000110 abs(n)

```
#include <iostream>
using namespace std;

int abs_no_branch(int n) {
   int mask = n >> 31;          // -1 si n<0, 0 si n>=0
   return (n ^ mask) - mask;   // truco del valor absoluto
}

int main() {
   cout << abs_no_branch(7) << endl;   // 7
   cout << abs_no_branch(-7) << endl;  // 7
   return 0;
}
```

📌 Sin ramificaciones (bit manipulation trick)

La idea es aprovechar el **bit de signo** en la representación en **complemento a dos**.
En casi todos los lenguajes y arquitecturas:

- El bit más significativo (MSB) indica el **signo**:
  - 0 → positivo
  - 1 → negativo

Podemos usar un **shift aritmético** para **propagar** ese bit de signo:
Shift aritmético a la derecha (**n >> 31** en 32 bits):

```
mask = n >> 31    // Si n ≥ 0 → llena con ceros → mask = 000...000 = 0.
                  // Si n < 0 → llena con unos (propaga el bit de signo) → mask = 111...111 = -1 en complemento a dos.
```

Luego aplicamos:
Fórmula: **abs(n)** = **(n ^ mask)** – mask

Caso A: n = 5

```
          n      = 00000000 00000000 00000000 00000101   (5)
mask   = n >> 31 = 00000000 00000000 00000000 00000000   (mask = 0, porque n>=0)

      (n ^ mask) = 00000000 00000000 00000000 00000101   (5)  XOR ^
resultado  = (n ^ mask) – mask  = 5-0=5
```

- **(5 ^ 0) - 0 = 5** ✅

Caso B: n = -5 (complemento a dos)

```
          n      = 11111111 11111111 11111111 11111011   (-5) Primero -5 en complemento a dos (32 bits):
mask   = n >> 31 = 11111111 11111111 11111111 11111111   (-1)


          n      =  11111111  11111111  11111111  11111011   (n = -5)
mask   = n >> 31  = 11111111  11111111  11111111  11111111   (mask = -1, porque n<0)
-----------------------------------
      (n ^ mask) = 00000000  00000000 00000000  00000100   (4) XOR ^

resultado = (n ^ mask) - mask = 4 - (-1) = 4 + 1 = 5
```

- **(-5 ^ -1) - (-1) = (4) + 1 = 5** ✅

La idea es usar la expresión (n+mask)^mask para calcular el valor absoluto de n, donde mask es n >> 31 (suponiendo almacenamiento de 32 bits para enteros). La máscara n >> 31 se evaluará como 0 para números positivos y -1 negativos.

¿Cómo funciona?
Para un **entero positivo n**, la máscara será **0 y (n + 0) ^ 0 = n**.
Para un **entero negativo n**, la máscara será **-1 y (n - 1) ^ -1 = n**.
¿Pero cómo?
Sabemos que, en complemento a 2, un **número negativo** se representa **invirtiendo** los **bits** de su contraparte positiva **y sumándolos 1**. Por tanto para **obtener la contraparte positiva(es decir el modulo)** de un número negativo, **réstale 1** e **invierta sus bits**. Es decir:
-n = ~n + 1
n = ~(-n – 1) //le resto 1, e invierto sus bits al negativo (-n)
nota: La idea es la misma: restamos una máscara -1 del número negativo dado e invertimos sus bits tomando su valor bit a bit XOR con la máscara

Solución alternativa
Si se permite la ramificación, podríamos haber utilizado directamente la expresión **((n >> 31) & 1) ? (~n + 1) : n** para devolver el valor absoluto de n.
Dado que **no se permite la ramificación**, podemos modificar fácilmente la expresión anterior a **((n >> 31) & (~n + 1)) | ((~n >> 31) & n)** la cual devuelve el valor absoluto de n sin usar el operador ternario. Como se mencionó anteriormente, la máscara **n >> 31** se evalúa como 0 (cero) para números positivos y -1(distinto de cero) para números negativos. De igual manera, la máscara **~n >> 31** se evalúa como 0(cero) para números negativos y -1(distinto de cero) para números positivos.
Ahora bien, para un número **positivo**, **(n >> 31) & (~n + 1)** será todo 0's y **(~n >> 31) & n** será igual a n, y para un número **negativo**, **(n >> 31) & (~n + 1)** será igual a n y **(~n >> 31) & n** será todo 0's. Por lo tanto, en cualquier caso, un lado del OR operador es 0, y el OR operador devolverá el valor distinto de cero, que resulta ser el valor absoluto de n.

Operaciones bit a bit con números enteros (int)

Obtener el entero máximo

```
int maxInt = ~(1 << 31);
int maxInt = (1 << 31) - 1;
int maxInt = (1 << -1) - 1;
```

Obtener el entero mínimo

```
int minInt = 1 << 31;
int minInt = 1 << -1;
```

Consigue el máximo largo

```
long maxLong = ((long)1 << 127) - 1;
```

Multiplicado por 2

```
n << 1; // n*2
```

Dividido por 2

```
n >> 1; // n/2
```

Multiplicado por la m-ésima potencia de 2

```
n << m; // (3<<5) ==>3 * 2^5 ==> 96
```

Dividido por la m-ésima potencia de 2

```
n >> m; // (20>>2) ==>(20/( 2^2) ==> 5
```

Comprobar número impar

```
(n & 1) == 1;
```

Intercambiar dos valores

```
a ^= b;
b ^= a;
a ^= b;
```

Obtenga valor absoluto

```
(n ^ (n >> 31)) - (n >> 31);
```

Obtenga el máximo de dos valores

```
b & ((a-b) >> 31) | a & (~(a-b) >> 31);
```

Obtener el mínimo de dos valores

```
a & ((a-b) >> 31) | b & (~(a-b) >> 31);
```

Comprueba si ambos tienen el mismo signo

```
(x ^ y) >= 0;
```

Calcular 2^n

```
2 << (n-1);
```

Si es factorial de 2

```
n > 0 ? (n & (n - 1)) == 0 : false;
```

Módulo 2^n contra m

```
m & (n - 1);
```

Obtenga el promedio

```
(x + y) >> 1;
```

```
((x ^ y) >> 1) + (x & y);
```

Obtener el bit m-ésimo de n (de menor a mayor)

```
(n >> (m-1)) & 1;
```

Establezca el bit m-ésimo de n en 0 (de bajo a alto)

```
n & ~(1 << (m-1));
```

n + 1

```
-~n
```

n-1

```
~-n
```

Obtenga el número de contraste

```
~n + 1;
(n ^ -1) + 1;
```

si (x==a) x=b; si (x==b) x=a;

```
x = a ^ b ^ x;
```

Calcular el signo de un entero		
Calcule el valor absoluto entero (abs) sin bifurcarse		
Calcule el mínimo (min) o el máximo (max) de dos números enteros sin bifurcarse		
Versiones rápidas y sucias:		
Determinar si un número entero es una potencia de 2		
Signo que se extiende desde un ancho de bits constante		
Signo que se extiende desde un ancho de bits variable		
Signo que se extiende desde un ancho de bits variable en 3 operaciones		
Establecer condicionalmente o borrar bits sin ramificación Negar condicionalmente un valor sin bifurcar		
Combinar bits de dos valores según una máscara		
Conjunto de bits de conteo (forma ingenua)		

Bits de conteo establecidos por la tabla de búsqueda		
Juego de conteo de bits, a la manera de Brian Kernighan		
Bits de conteo establecidos en palabras de 14, 24 o 32 bits usando instrucciones de 64 bits		
Conjunto de bits de conteo, en paralelo		
Cuente los bits establecidos (rango) desde el bit más significativo hasta una posición dada		
Seleccione la posición del bit (desde el bit más significativo) con el conteo (rango) dado		
Cálculo de la paridad de forma ingenua		
Calcular paridad por tabla de búsqueda		
Calcule la paridad de un byte usando multiplicación de 64 bits y división de módulo		
Calcular la paridad de palabra con una multiplicación		
Calcular la paridad en paralelo		
Intercambio de valores con resta y suma		
Intercambio de valores con XOR		
Intercambio de bits individuales con XOR		
Bits inversos de la manera obvia		
Bits inversos en palabra por tabla de búsqueda		
Invierta los bits en un byte con 3 operaciones (multiplicación de 64 bits y división de módulo):		
Invierta los bits en un byte con 4 operaciones (multiplicación de 64 bits, sin división):		

Invierta los bits en un byte con 7 operaciones (sin 64 bits):		
Invierta una cantidad de N bits en paralelo en operaciones de 5 * lg(N):		
Calcule la división del módulo por 1 << s sin un operador de división		
Calcule la división del módulo por (1 << s) - 1 sin un operador de división		
Calcule la división del módulo por (1 << s) - 1 en paralelo sin un operador de división		
Encuentre la base logarítmica 2 de un número entero con el MSB N establecido en operaciones O (N) (la forma obvia)		
Encuentre el registro de enteros base 2 de un entero con un flotante IEEE de 64 bits		
Encuentre la base logarítmica 2 de un número entero con una tabla de búsqueda		
Encuentre el logaritmo en base 2 de un entero de N bits en operaciones O(lg(N))		
Encuentre la base logarítmica 2 de un entero de N bits en operaciones O(lg(N)) con multiplicación y búsqueda		
Encuentra la base de registro de números enteros 10 de un número entero		
Encuentre la base de registro de enteros 10 de un entero de la manera obvia		
Encuentre la base de registro de enteros 2 de un flotador IEEE de 32 bits		
Encuentre la base de registro de enteros 2 de la raíz pow (2, r) de		

un flotante IEEE de 32 bits (para el entero r sin signo)		
Cuente los bits cero consecutivos (traseros) a la derecha linealmente		
Cuente los bits cero consecutivos (traseros) a la derecha en paralelo		
Cuente los bits cero consecutivos (traseros) a la derecha mediante búsqueda binaria		
Cuente los bits cero consecutivos (traseros) a la derecha convirtiéndolos en un flotador		
Cuente los bits cero consecutivos (traseros) a la derecha con división de módulo y búsqueda		
Cuente los bits cero consecutivos (traseros) a la derecha con multiplicar y buscar		
Redondee a la siguiente potencia más alta de 2 mediante lanzamiento flotante		
Redondea a la siguiente potencia más alta de 2		
Intercalar bits de forma obvia		
Intercalar bits por búsqueda de tabla		
Intercalar bits con multiplicación de 64 bits		
Intercalar bits por números mágicos binarios		
Determinar si una palabra tiene un byte cero		
Determinar si una palabra tiene un byte igual a n		
Determinar si una palabra tiene un byte menor que n		
Determinar si una palabra tiene un byte mayor que n		
Determinar si una palabra tiene un byte entre m y n		

Calcule la siguiente permutación de bits lexicográficamente		

https://www.geeksforgeeks.org/compute-modulus-division-by-a-power-of-2-number/?ref=lbp

Problemas sencillos sobre manipulaciones de bits y algoritmos bit a bit

1- Dado un número, encuentre el mayor número menor que el número dado que sea potencia de dos o encuentre el número de bits más significativo. Ejemplo: *El mayor número que es una potencia de 2 menor que 10 es 8*	`Msb = parseInt(Math.log2(n))` `return (1 << msb);`	
2- Posición del bit establecido más a la derecha	bit = parseInt(Math.log2(n & -n)+1)	
3- Obtener la posición del bit desactivado más a la derecha	bit=parseInt(Math.log2(~n & -~n)+1)	
4- Compruebe si el K-th bit está establecido o no	(n & (1<<(k)))!=0	
5- Compruebe si el bit en la posición dada está activado o desactivado	(n & (1<<(k-1)))!=0	
6- Encuentra la posición del único bit establecido		
Comprobar el desbordamiento de enteros	https://www.similaranswer.es/como-se-puede-producir-el-desbordamiento/#que-causa-el-desbordamiento-aritmetico	

Encuentre XOR de dos números sin usar el operador XOR	xor = or - and = ((x | y) - (x & y))	

Comprobar si dos números son iguales sin usar operadores aritméticos y de comparación	if ((x ^ y) != 0) {
	return('diferentes');
	} else {
	return('iguales');
	}	

Detectar si dos números enteros tienen signos opuestos	return ((x ^ y) < 0);	
¿Cómo intercambiar dos números sin usar una variable temporal?	a = a ^ b b = a ^ b a = a ^ b	
12- Campesino Ruso (Multiplica dos números usando operadores bit a bit)		

Problemas mediano sobre manipulaciones de bits y algoritmos bit a bit

Intercambiar bits en un número dado		
El menor de tres enteros sin operadores de comparación		
Calcule el mínimo o el máximo de dos números enteros sin bifurcarse		
La potencia más pequeña de 2 mayor o igual que n		
Programa para encontrar la paridad		
Comprobar si la representación binaria de un número es palíndromo		
Generar códigos grises de n bits		
Comprobar si un número dado es escaso o no		
Algoritmo de Euclides cuando las operaciones % y / son costosas		
Calcula el cuadrado de un número sin usar *, / y pow()		

Copiar bits establecidos en un rango		
Comprobar si un número es sombrío		
Conversión de gris a binario y de binario a gris		

Problemas dificiles sobre manipulaciones de bits y algoritmos bit a bit

Siguiente número más alto con el mismo número de bits establecidos		
Encuentre el subarreglo máximo XOR en un arreglo dado		
Encuentre la secuencia más larga de 1 en representación binaria con un giro		
Números más pequeños y más grandes más cercanos (o siguientes) con el mismo número de bits establecidos		
Enmascaramiento de bits y programación dinámica | Problema del vendedor ambulante		
Calcule la paridad de un número usando XOR y búsqueda de tabla		
Cifrado XOR cambiando el texto sin formato		
Cuente los pares en una matriz que tienen al menos un dígito común		
Programa de Python para convertir flotante a binario		
Algoritmo de multiplicación de Booth		
Número de pares con Concatenación Pandigital		
Encuentra el n-ésimo número cuya representación binaria es un palíndromo		

Encuentre los dos elementos que no se repiten en una matriz de elementos que se repiten/Números únicos 2		

https://www.geeksforgeeks.org/find-two-non-repeating-elements-in-an-array-of-repeating-elements/?ref=lbp

Algoritmos bit a bit

https://www.geeksforgeeks.org/bitwise-algorithms/?ref=lbp

Estructura de datos tipo abstractos

https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/?utm_campaign=DSA%20Page%20Tracker&utm_content=Course%20Page&utm_id=DSA-Page-Tracker&utm_medium=Landing%20Page%20Click&utm_source=Website&utm_term=DSA%20Page%20Promo